

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

*I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

SIMATS ENGINEERING

Assessment Tool 1 – Application Based Questions
Computer Architecture (CSA12)

Name	G. Naveen Kumar Reddy
Register No.	192525288
Course Outcome	CO1 (BL4) – CPU, Memory, Bus Structures and I/O Components

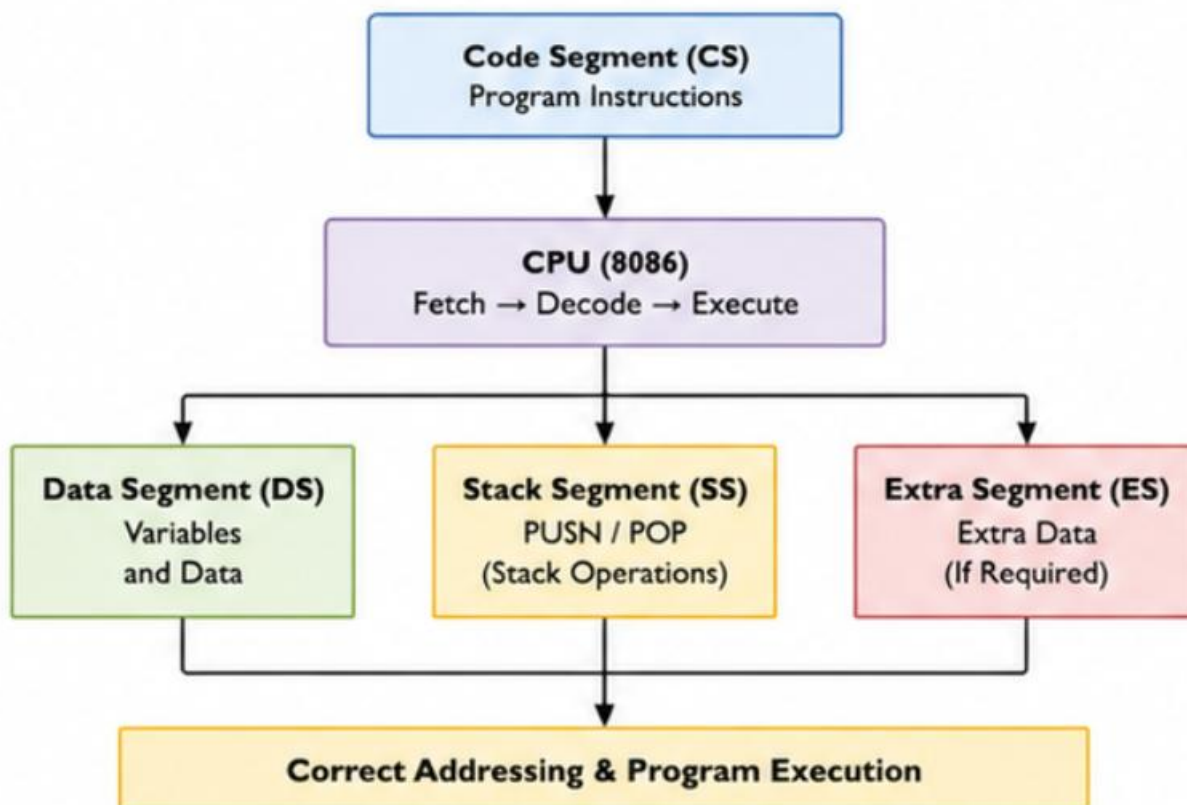
Q4. 8086 Segmented Memory System

Theory

An 8086 microprocessor uses segmented memory to organize programs and data efficiently. Memory is divided into Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). The CPU accesses instructions from the Code Segment, stores variables in the Data Segment, and manages function calls and interrupts using the Stack Segment. During multitasking, incorrect segment register values or improper stack handling can cause incorrect data access, stack overflow, and program crashes. Correct instruction execution requires proper loading of segment registers and the use of suitable addressing modes such as immediate, register, direct, indirect, based, indexed, and based-indexed addressing to ensure accurate memory access.

Schematic Diagram

**Q4. 8086 Segmented Memory System
Memory Organization & Execution Flow**



Example Code

```
</> asm

MOV AX, DATA
MOV DS, AX
MOV AL, 10H
ADD AL, 20H
MOV RESULT, AL
INT 21H
```

Advantages

- Efficient memory organization
- Supports multitasking programs
- Faster instruction execution
- Easy management of code, data, and stack
- Improves program reliability

Conclusion

Proper management of code, data, and stack segments, along with correct addressing modes, ensures accurate instruction execution, prevents incorrect data access and stack overflow, and improves overall system performance.

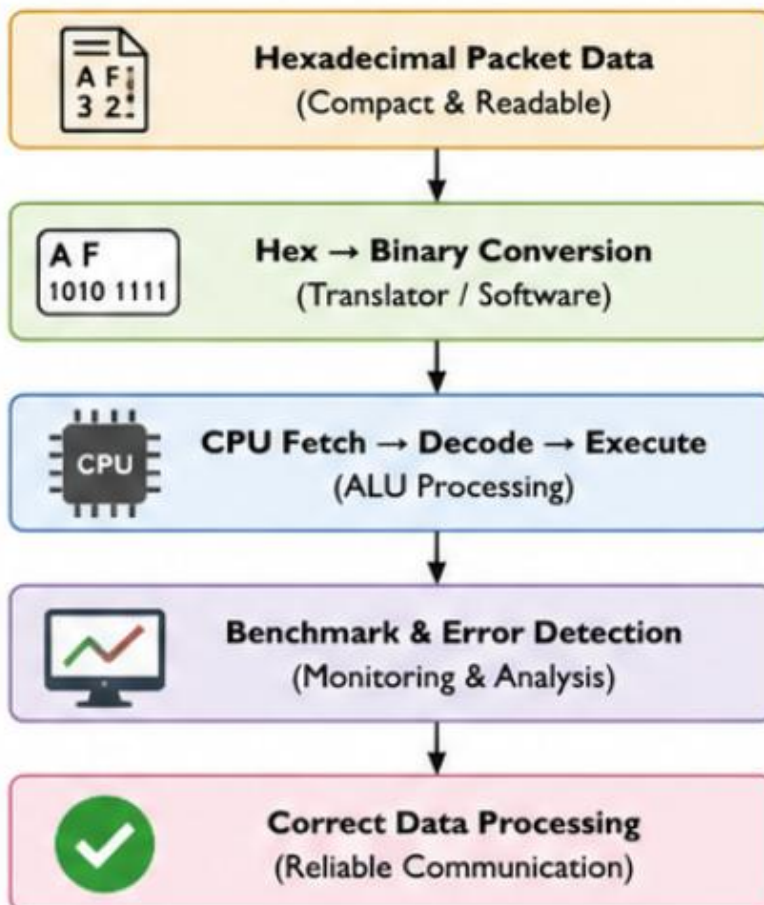
Q5. Number System Conversion and CPU Performance

Theory

Data communication systems commonly represent packet information in **hexadecimal format** because it is compact and easy to read. Before processing, hexadecimal values are converted into **binary**, which is the language understood by the CPU. Incorrect conversion between hexadecimal and binary can lead to **wrong instruction decoding, incorrect data processing, and communication errors**. Efficient CPU design, including optimized ALU operations, pipelining, cache memory, and benchmarking techniques, helps detect processing errors, improve execution speed, and ensure reliable real-time communication performance.

Schematic Diagram

Q5. Number System Conversion & CPU Performance
Data Processing Flow



Example Code

```
</> C

#include <stdio.h>

int main() {
    int n;
    scanf("%x", &n);
    printf("%d", n);
    return 0;
}
```

Advantages

- Simplifies packet representation
- Enables accurate binary processing
- Improves CPU performance
- Detects processing errors quickly
- Enhances real-time system reliability

Conclusion

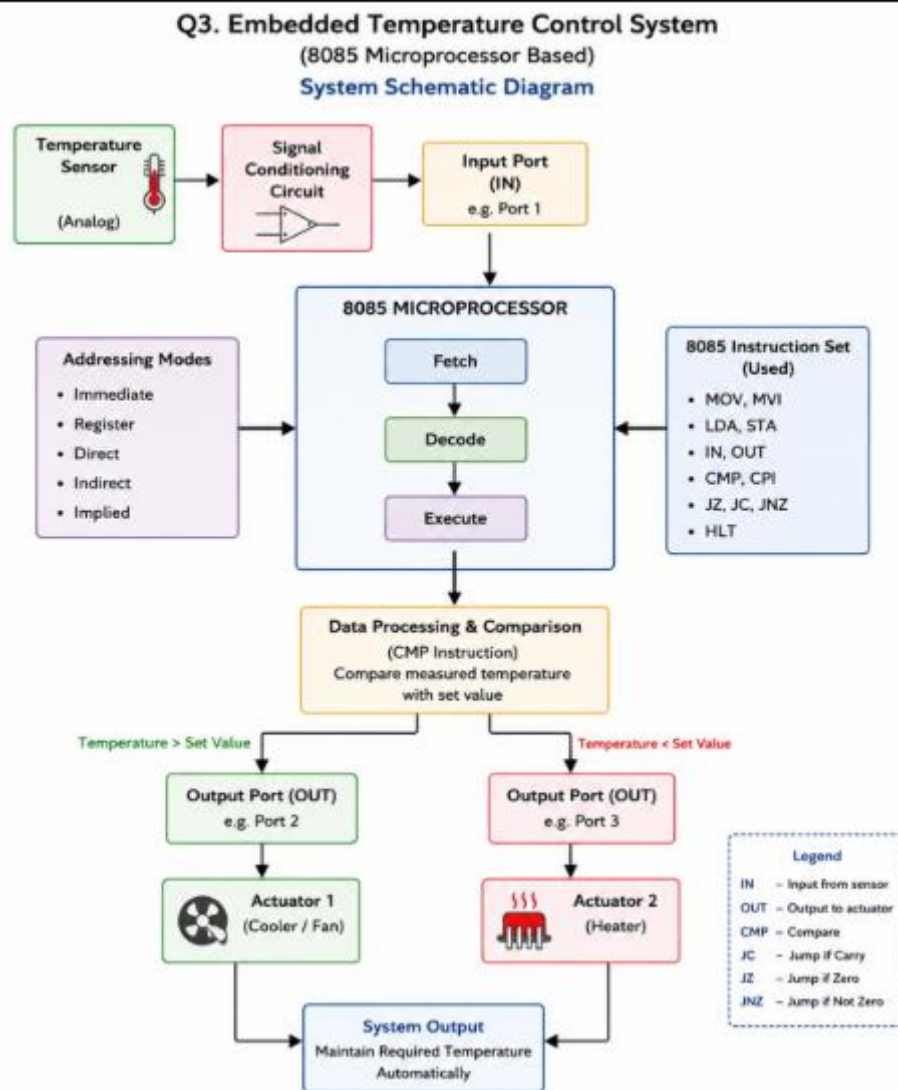
Accurate **hexadecimal-to-binary conversion**, efficient **CPU architecture**, and effective **benchmarking** ensure correct instruction execution, minimize processing errors, and improve the performance and reliability of real-time communication systems.

Q3. Embedded Temperature Control System (8085 Microprocessor)

Theory

An 8085-based embedded temperature control system receives temperature values from sensors through input devices and processes them using the CPU. The processor accesses data using different addressing modes such as immediate, register, direct, indirect, and implied addressing. If the wrong addressing mode is used, the processor may read incorrect memory locations or register values, resulting in wrong temperature readings and improper actuator operation. Using the correct addressing mode and suitable 8085 instructions like MOV, MVI, LDA, STA, IN, OUT, CMP, and conditional jump instructions (JZ, JC, JNZ) ensures accurate data processing and reliable control.

Schematic Diagram



Example Code

```
</> asm
```

```
IN 01H  
CPI 50H  
JC COOL  
OUT 02H  
HLT
```

```
COOL :  
OUT 03H  
HLT
```

Advantages

- Accurate sensor data access
- Reliable temperature control
- Efficient memory utilization
- Faster instruction execution
- Reduced processing errors

Conclusion

Proper use of 8085 addressing modes and instruction set ensures accurate temperature measurement, reliable actuator control, and efficient operation of embedded industrial system.

